

Prueba Técnica — Sistema de Agendamiento de Citas Médicas

Fecha: 2026-06-10 **Tecnología:** Java con Spring Boot 3.x **Plazo de entrega:** 3 días

Contexto del Problema

MediSalud es una clínica que está digitalizando su proceso de agendamiento de citas médicas. Actualmente, los pacientes llaman por teléfono para reservar turnos, lo que genera conflictos de horarios, duplicidad de citas y falta de control sobre las cancelaciones.

La dirección ha encargado un **sistema de agendamiento digital** que permita a los pacientes reservar citas con médicos disponibles, controlar la disponibilidad por franja horaria, y registrar cancelaciones con penalizaciones para reducir el ausentismo.

El sistema debe ser lo suficientemente robusto como para escalar en el futuro, pero para esta prueba se requiere un MVP funcional con buenas prácticas de desarrollo.

Importante: Este es un sistema de backend (API REST). No se requiere frontend.

Requerimientos Funcionales

RF-01: Registro de Médicos

El sistema debe permitir registrar médicos con la siguiente información:

- **ID** (identificador único, auto-generado o UUID)
- **Nombre completo** (obligatorio, 3-100 caracteres)
- **Especialidad** (obligatorio, ej: "Cardiología", "Pediatría", "Dermatología")
- **Teléfono** (opcional, formato mínimo 7 dígitos)

- **Email** (opcional, formato email válido)

Datos de ejemplo para carga inicial:

ID	Nombre	Especialidad	Teléfono	Email
1	Dra. María González	Cardiología	555-1001	maria.gonzalez@medisalud.com
2	Dr. Carlos Ruiz	Pediatría	555-1002	carlos.ruiz@medisalud.com
3	Dra. Ana López	Dermatología	555-1003	ana.lopez@medisalud.com

RF-02: Registro de Pacientes

El sistema debe permitir registrar pacientes:

- **ID** (identificador único, auto-generado o UUID)
- **Nombre completo** (obligatorio, 3-100 caracteres)
- **Documento de identidad** (obligatorio, único en el sistema, mínimo 7 caracteres)
- **Teléfono** (obligatorio, mínimo 7 dígitos)
- **Email** (obligatorio, formato email válido)

RF-03: Reserva de Citas

El sistema debe permitir a un paciente reservar una cita con un médico:

- **ID** (identificador único, auto-generado)
- **Paciente** (referencia al paciente registrado)
- **Médico** (referencia al médico registrado)
- **Fecha y hora** (formato ISO 8601, obligatorio)
- **Estado** (inicialmente "PROGRAMADA")

RF-04: Consulta de Citas Disponibles

El sistema debe permitir consultar las citas disponibles de un médico en un rango de fechas:

- Recibir como parámetros: `medicoId` , `fechaInicio` , `fechaFin`
- Devolver las franjas horarias disponibles (cada franja = 30 minutos)
- Las franjas ya ocupadas por citas no deben aparecer en el resultado

RF-05: Cancelación de Citas

El sistema debe permitir cancelar una cita programada:

- Recibir el ID de la cita
- Cambiar su estado a "CANCELADA"
- Registrar la fecha/hora de cancelación
- Aplicar la regla de penalización por cancelación (ver RN-05)

RF-06: Listado de Citas

El sistema debe permitir listar citas con los siguientes filtros opcionales:

- Por `medicoId`
- Por `pacienteId`
- Por `estado` (PROGRAMADA, CANCELADA, ATENDIDA)
- Por rango de fechas (`fechaInicio` , `fechaFin`)

Reglas de Negocio

RN-01: Franjas Horarias de Atención

Los médicos solo atienden en horarios laborales:

- **Lunes a Viernes:** 08:00 - 18:00
- **Sábados:** 08:00 - 13:00
- **Domingos y festivos:** No hay atención

Las citas solo pueden programarse en franjas de **30 minutos** dentro de estos horarios.

RN-02: No Duplicidad de Citas

Un médico **no puede tener dos citas** en la misma franja horaria de 30 minutos. Si un paciente intenta agendar una cita en un horario ya ocupado, el sistema debe rechazarla con un error claro.

RN-03: Antigüedad Mínima del Paciente

Solo se pueden agendar citas para pacientes con **edad mínima de 0 años** (no se aceptan fechas de nacimiento futuras). La validación de edad no es obligatoria en el registro, pero sí al agendar: si no se proporciona fecha de nacimiento, se asume edad 0.

RN-04: Conflicto de Paciente

Un paciente **no puede tener dos citas** con el mismo médico en la misma franja horaria. Si un paciente ya tiene una cita programada con un médico en un horario, no puede agendar otra cita con el mismo médico en ese horario (aunque sea otro médico en otra especialidad).

RN-05: Penalización por Cancelación Tardía

Si una cita es cancelada con **menos de 2 horas** de antelación a la hora programada, se registra una penalización para ese paciente. Un paciente con **3 o más penalizaciones** en los últimos 30 días **no puede agendar nuevas citas** hasta que transcurran esos 30 días.

RN-06: Reprogramación

Una cita puede ser reprogramada (cambiar fecha/hora). La reprogramación implica:

1. Cancelar la cita anterior (aplicando RN-05 si corresponde)
 2. Crear una nueva cita con el nuevo horario
 3. Validar que el nuevo horario esté disponible (RN-02 y RN-04)
-

Datos de Referencia

Franjas Horarias de Ejemplo

Para un médico disponible, las franjas de 30 minutos en un día laboral son:

Franja	Horario
1	08:00 - 08:30
2	08:30 - 09:00
...	...
20	17:30 - 18:00

En sábado (hasta 13:00): franjas 1-10 (08:00 - 13:00).

Requerimientos Técnicos

- La **arquitectura es de libre elección** del candidato. Se evaluará la decisión arquitectónica como indicador de capacidades de diseño.
 - El **acceso a datos es de libre elección**: puede usar JDBC, JPA/Hibernate, MyBatis, o cualquier otro enfoque. Incluso puede usar una base de datos en memoria (H2, SQLite) si lo justifica en la documentación.
 - Se requieren **pruebas automatizadas** (unitarias como mínimo).
 - Se debe manejar **errores de forma consistente** con respuestas HTTP apropiadas (200, 201, 400, 404, 409, 500).
 - El proyecto debe compilar y ejecutarse localmente con `java -jar` o `mvn spring-boot:run` / `gradle bootRun`.
-

Entregables Obligatorios

1. **Repositorio GitHub público** con el código fuente completo

2. **README.md** con:

- Instrucciones claras para ejecutar el proyecto localmente
- Descripción de la arquitectura elegida y justificación
- Tecnologías utilizadas
- Endpoints de la API con ejemplos de request/response

3. **Tests automatizados** que validen los flujos de negocio

4. **El despliegue en nube es valorado positivamente** como diferenciador

Criterios de Evaluación

Se evaluará:

- **Cumplimiento de los requerimientos funcionales y reglas de negocio** — ¿El sistema funciona como se espera?
 - **Arquitectura y diseño de la solución** — ¿La arquitectura elegida es coherente? ¿Hay separación de responsabilidades?
 - **Calidad del código y principios de diseño aplicados** — SOLID, DRY, nombres significativos, coherencia
 - **Manejo de errores y excepciones** — ¿Los errores se manejan de forma consistente? ¿Se devuelven mensajes claros?
 - **Seguridad de la aplicación** — Validación de inputs, protección contra inyección, manejo de datos sensibles
 - **Cobertura y calidad de las pruebas** — ¿Los tests cubren los flujos críticos? ¿Cubren casos borde?
 - **Documentación** — ¿El README es claro? ¿La arquitectura está documentada y justificada?
-

Notas Adicionales

- Puedes usar herramientas de IA (Copilot, ChatGPT, etc.) para desarrollar la prueba. Se asume que el candidato las utilizará.

- El diferenciador entre niveles será la **calidad estructural**: arquitectura, principios de diseño, manejo de errores, patrones aplicados, y la profundidad de las pruebas.
 - No se requiere frontend — solo backend (API REST).
 - No se requiere autenticación/autorización para esta prueba.
-